
MULTILEVEL CACHE HIERARCHIES AND SYSTEM DESIGN

Cache Performance Optimization

Topics:

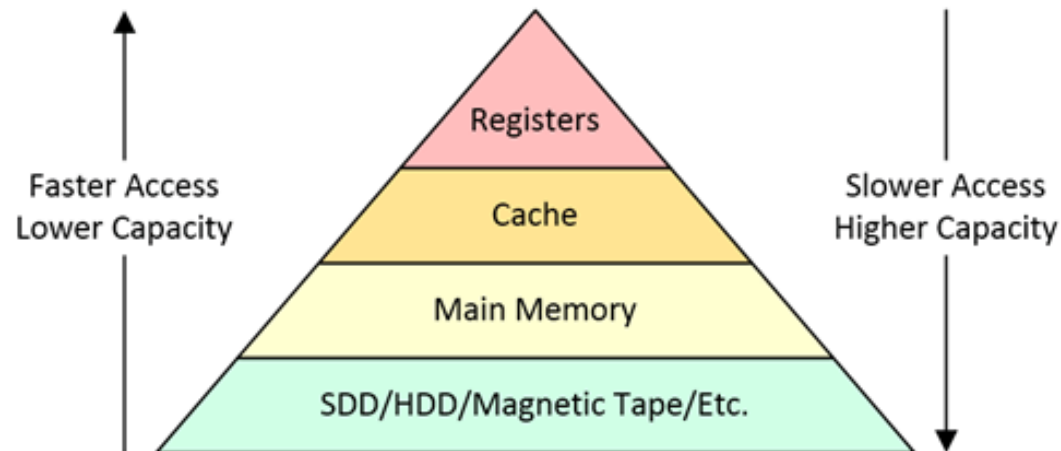
- Why multiple cache levels exist
- How AMAT extends to multiple levels
- Roles of L1, L2 and L3
- Inclusive vs exclusive hierarchies
- Private vs shared caches

Why Single Level is not Enough

Trade-off:

- Small cache -> fast but many misses
- Large cache -> slow but fewer misses

We cannot optimize both simultaneously!



Multilevel Hierarchy

Introduce:

- L1 -> very fast, small
- L2 -> medium
- L3 (LLC) -> large, slower

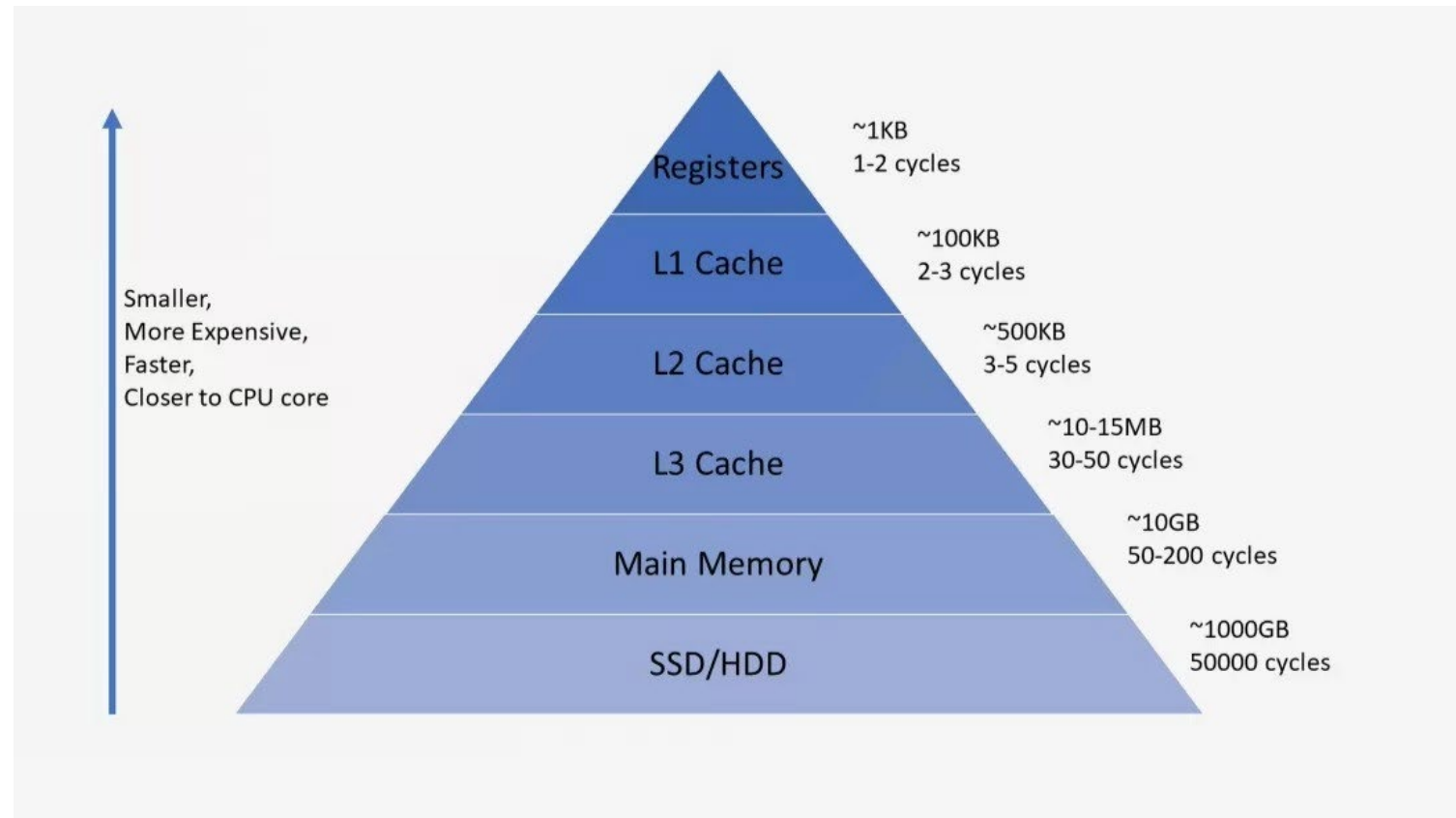
The idea is that each level reduces the effective miss penalty

Multilevel Access Flow

Access flow:

1. Check L1
2. Miss -> check L2
3. Miss -> check L3
4. Miss -> Dram

Each level adds latency



AMAT for Two Levels

The generic AMAT equation is:

$$AMAT = HitTime + MissRate \times MissPenalty$$

This equation can be specified for the two levels scenario:

$$AMAT = HT_{L1} + MR_{L1} \times (HT_{L2} + MR_{L2} \times MP_{L2})$$

Interpretation: L2 reduces effective penalty of L1 misses

AMAT for Two Levels

Example:

L1 HT = 1, MR = 5%

L2 HT = 10, MR = 20%

Dram penalty = 100

$$\begin{aligned} AMAT &= HT_{L1} + MR_{L1} \times (HT_{L2} + MR_{L2} \times MP_{L2}) = \\ &= 1 + 0.05 \times (10 + 0.20 \times 100) = 2.5 \end{aligned}$$

Without L2 we have:

$$\begin{aligned} AMAT &= HitTime + MissRate \times MissPenalty = \\ &= 1 + 0.05 \times 100 = 6 \end{aligned}$$

Role of Each Level

L1 Cache

- Extremely fast, small, on critical path

L2 Cache

- Reduces L1 miss penalty, moderately fast

L3 (LLC)

- Reduces DRAM accesses, large capacity

Level	Priority
L1	Latency
L2	Balance
L3	Capacity

The Problem: Conflict Misses in L1

Even with good cache size and low latency L1 (often low associativity e.g. 4-8 ways) still suffers from conflict misses

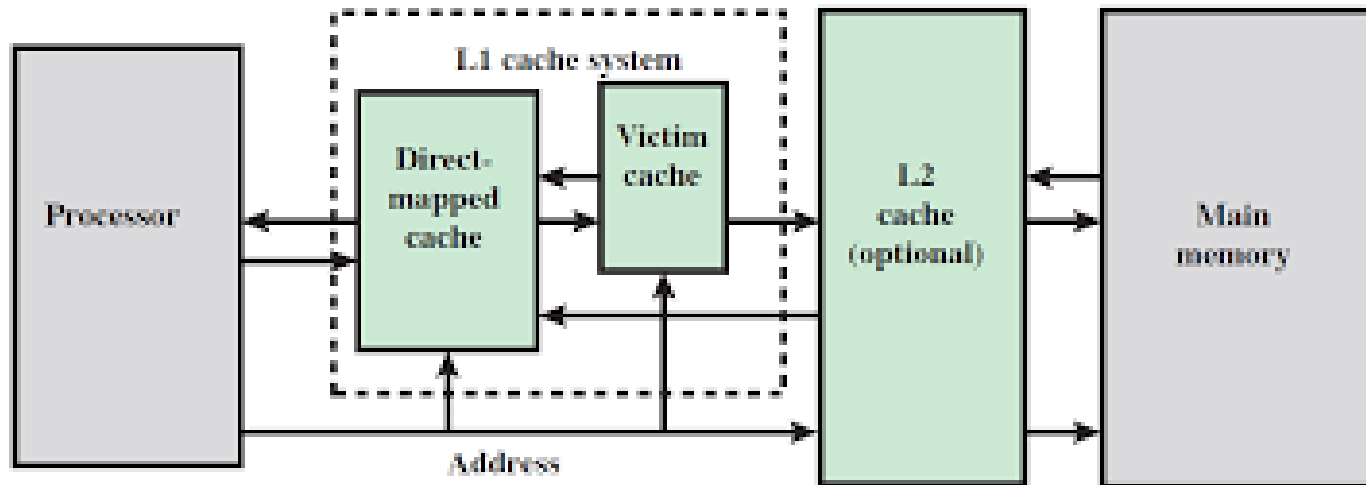
Solution -> Victim cache, a small fully-associative buffer placed between L1 and L2

Purpose -> store recently evicted cache lines from L1

Victim Cache

Flow:

1. L1 miss
2. Check victim cache
3. If hit -> swap with L1
4. If miss -> go to L2

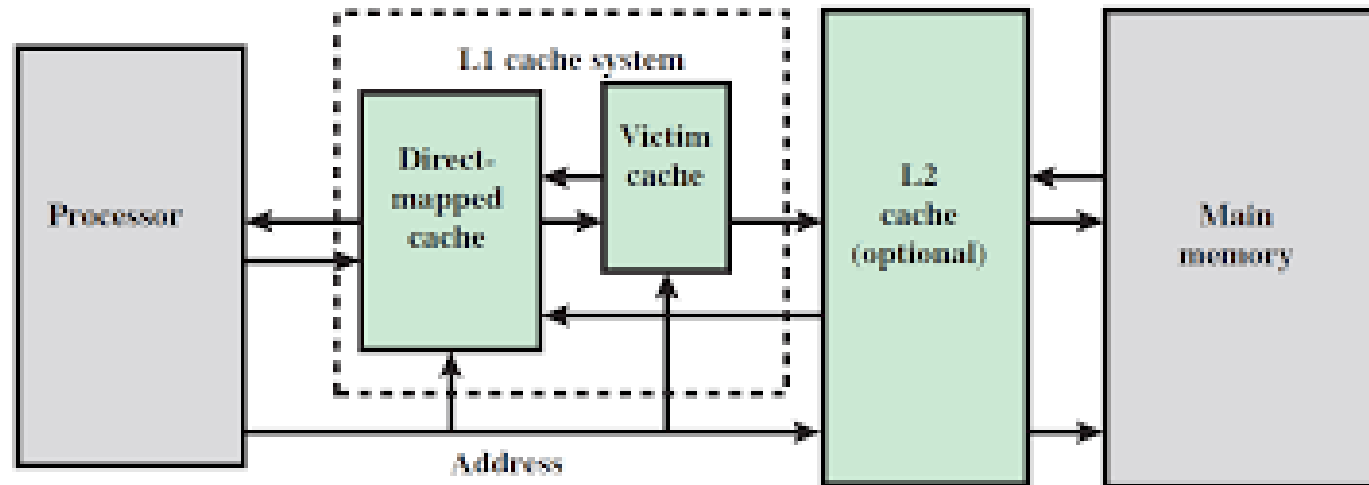


Key Mechanism: Swap

Victim cache uses swap, not copy!

If hit in victim cache:

- Bring block back to L1
- Move evicted L1 block into victim cache



Victim Cache - Example

A processor has 32 bit addresses and a 256 lines direct-mapped cache a 4 lines victim cache, each with a 16 B block. Compute the total misses of the following assembly code.

```
        la $t0, 0xC0ABD030
        la $t1, 0xD8EFC030
        li $t2, 32
        li $t3, 0
loop:    lbu $t4, 0($t0)
        lbu $t5, 0($t1)
        addiu $t0, $t0, 1
        addiu $t1, $t1, 1
        addu $t3, $t4, $t5
        addiu $t3, $t3, 1
        bne $t3, $t2, loop
```

Victim Cache - Example

L1 cache

	validity bit	tag	data
0	0		
1	0		
2	0		
3	0		
4	0		
254	0		
255	0		

Victim cache

	validity bit	tag	data
0	0		
1	0		
2	0		
3	0		

Victim Cache - Example

First load: `lbu $t4,0($t0)`

`$t0` contains the address `0xC0ABD030` -> offset = 0

cache index = 03

tag = C0ABD

Compulsory miss and caching of the first 16 B of vector A

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	C0ABD	16 B of array A
4	0		

.

254	0		
255	0		

L1 cache

Victim cache

	validity bit	tag	data
0	0		
1	0		
2	0		
3	0		

Victim Cache - Example

Second load: `lbu $t5,0($t1)`

`$t1` contains the address `0xD8EFC030` -> offset = 0

cache index = 03

tag = D8EFC

Conflict miss and caching of the first 16 B of vector B, evicted block moved to victim cache

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	D8EFC	16 B of array B
4	0		

L1 cache

.

254	0		
255	0		

Victim cache

	validity bit	tag	data
0	1	C0ABD	16 B of array A
1	0		
2	0		
3	0		

Victim Cache - Example

third load: `lbu $t4,0($t0)`

`$t1` contains the address `0xC0ABD031` -> offset = 1

cache index = 03

tag = C0ABD

Conflict miss but hit in victim cache -> swap L1 and victim

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	C0ABD	16 B of array A
4	0		

.

254	0		
255	0		

L1 cache

Victim cache

	validity bit	tag	data
0	1	D8EFC	16 B of array B
1	0		
2	0		
3	0		

Victim Cache - Example

The loop iterates over 32 iterations

At the first iteration there are 2 misses

The victim cache avoids conflict misses at the next iterations

At the 16th iteration we have 2 compulsory misses (block size is 16 B, array A and B are 32 B each)

No conflict misses!

Miss rate = $4/64 = 6.25\%$

What type of Misses Does it Fix?

Victim cache targets conflict misses

It does not help with

- Compulsory misses
- Capacity misses

Why fully associative?

Victim cache is small (4-16 entries) and fully associative to maximize probability of hit and to feature a low overhead due to small size

Performance Impact

Benefits:

- ✓ reduces conflict misses
- ✓ small hardware cost
- ✓ no impact on L1 hit time

Costs:

- ✗ extra lookup
- ✗ slightly higher complexity

Most effective when:

- L1 is low associativity (direct mapped or few ways)
- Access patterns cause index conflicts
- Working set fits in L1 + victim cache

Historical and Practical Use

Used in:

- Early high-performance CPUs
- Some embedded systems
- Specialized architectures

Less common today because L1 associativity increased (4-8 ways)

Still very relevant conceptually.

Inclusive Cache Hierarchy

All data in L1 is also present in L2 (and L3 and the other levels)

- ✓ Easy coherence
- ✓ Simple eviction tracking
- ✗ Redundant data
- ✗ Wasted capacity

Exclusive Cache Hierarchy

Data exists only in one level

✓ Maximizes total capacity

✓ No duplication

✗ More complex

✗ Harder coherence

Aspect	Inclusive	Exclusive
Capacity	Lower	Higher
Complexity	Lower	Higher
Coherence	Easier	Harder

Private and Shared Caches

In multicore systems we can have private and shared caches.

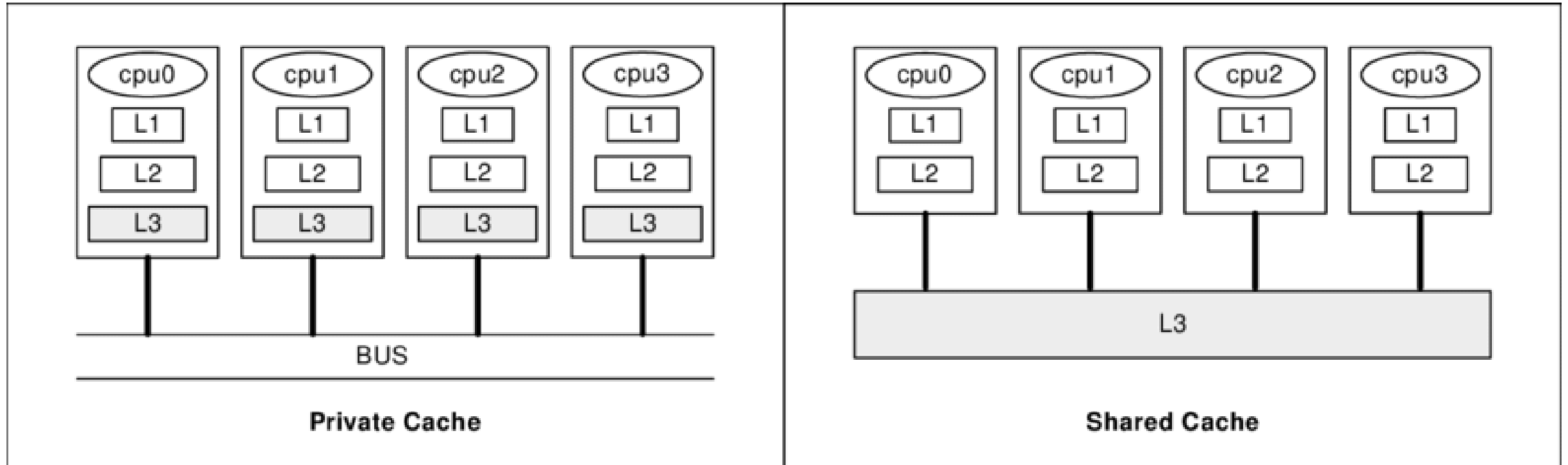
Private cache -> each core had its own cache

- ✓ low latency
- ✓ no contention
- ✗ duplication

Shared cache (LLC) -> each core accesses the same cache

- ✓ better utilization
- ✓ easier communication
- ✗ contention
- ✗ higher latency

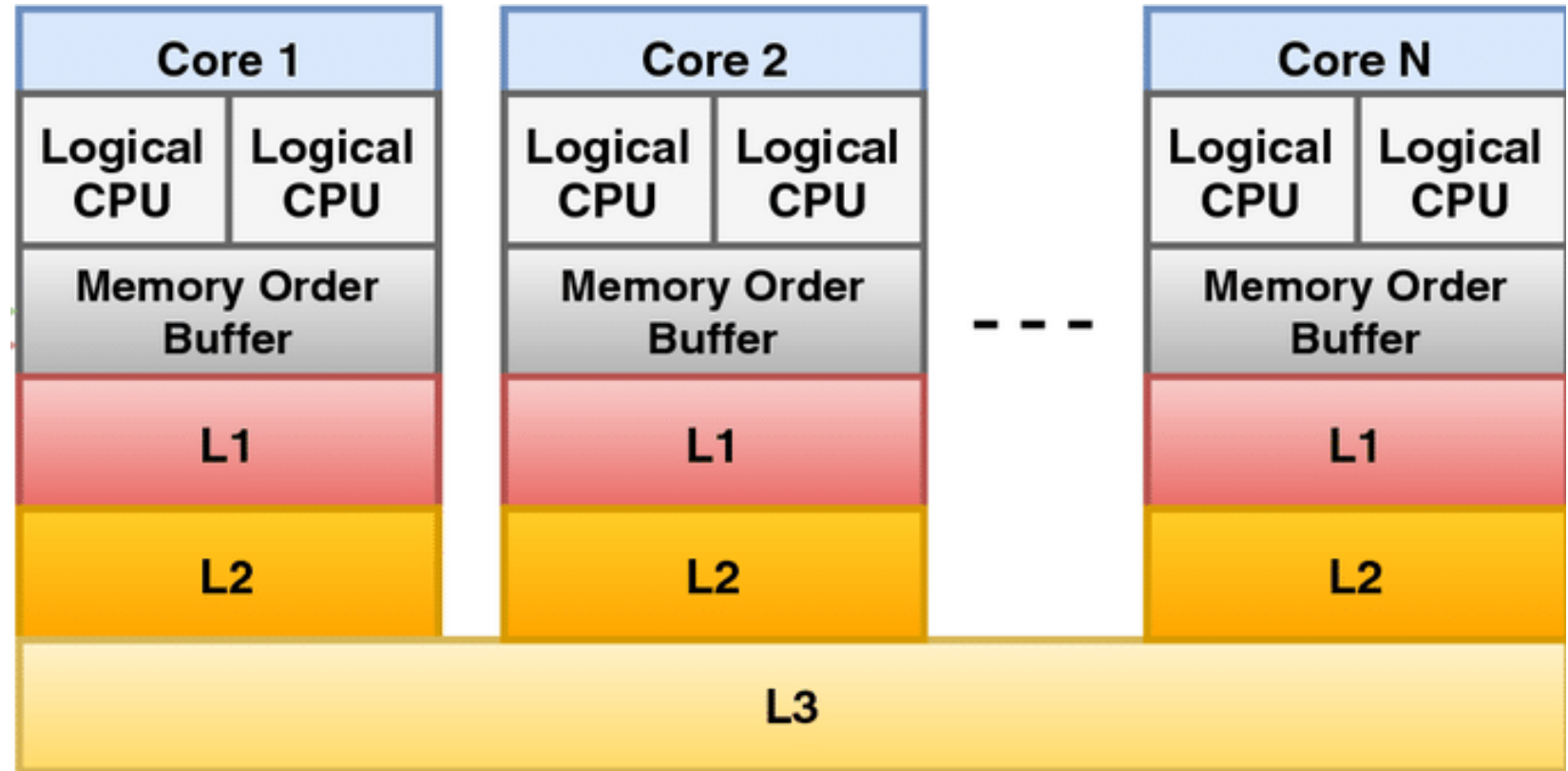
Private and Shared Caches



Modern CPU Design

Typical:

- L1 private
- L2 private
- L3 shared



Interconnection Matters

Caches are connected via:

- Ring bus
- Mesh
- Crossbar

Latency depends on physical distance

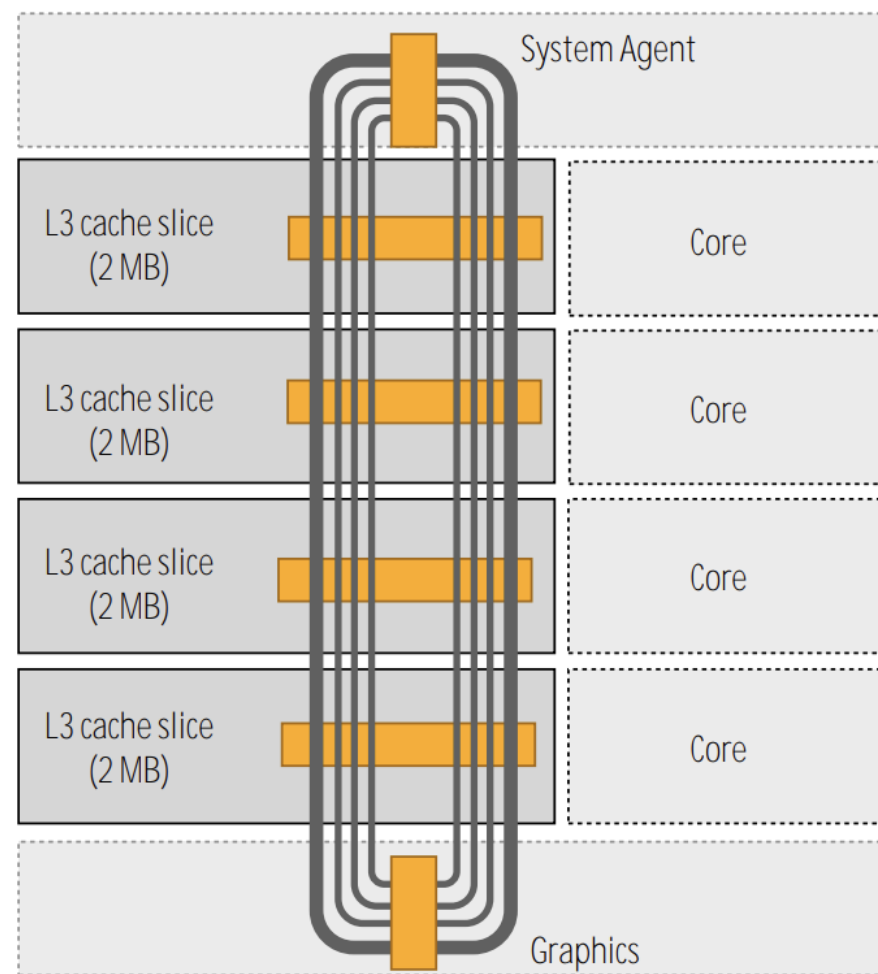
As cores increase:

- Coherence traffic increases
- LLC contention increases
- Latency variability increases

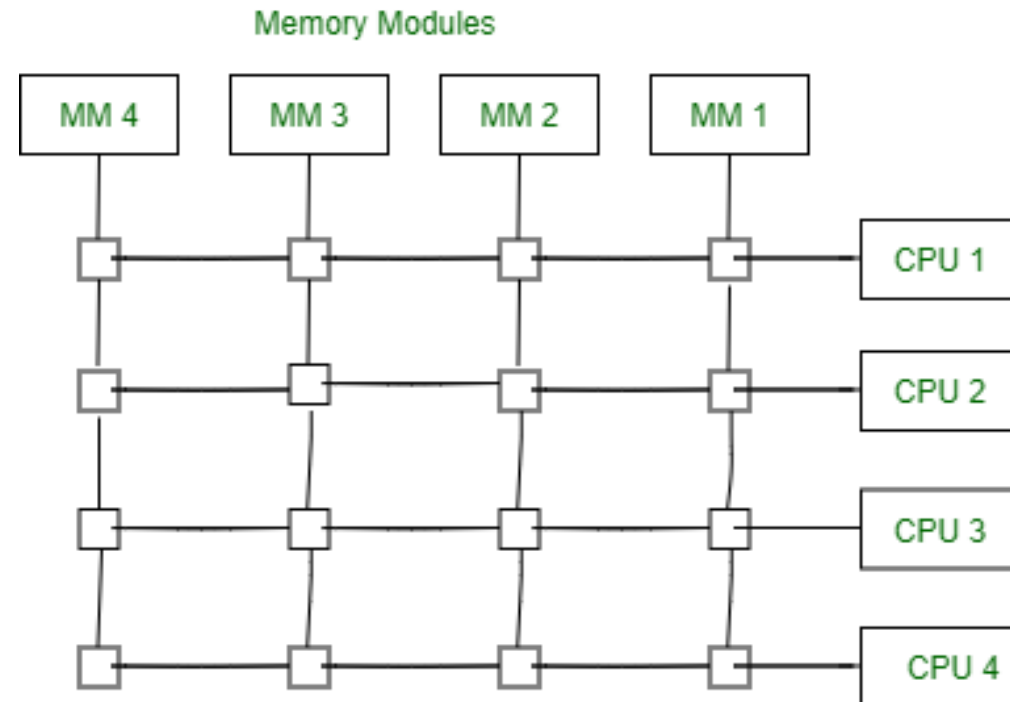
Memory becomes a shared bottleneck!

Interconnection Matters

Intel's Ring Interconnect

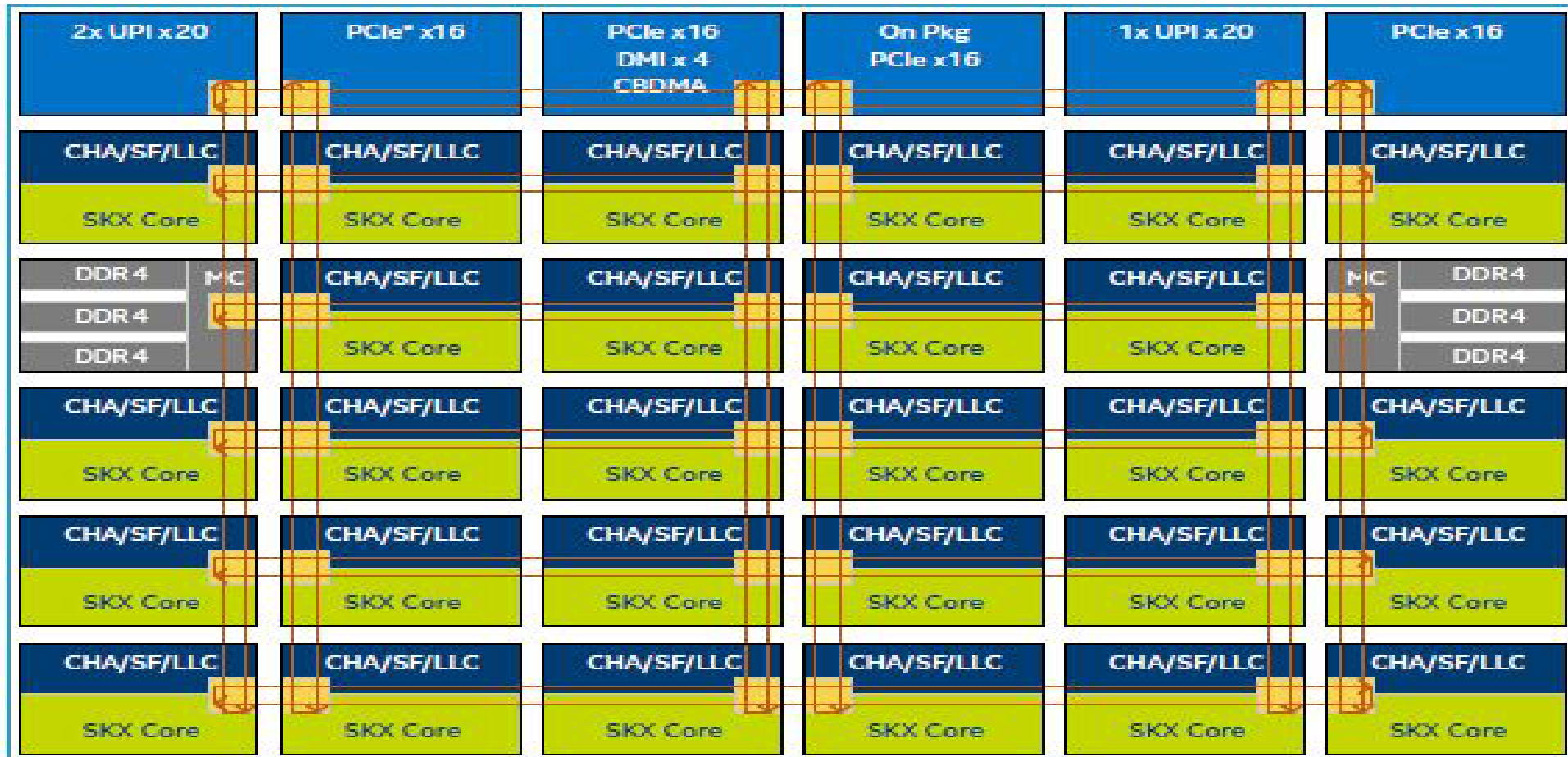


Crossbar Interconnect



Interconnection Matters

Intel's Mesh Interconnect



Engineering Trade-Offs

Design decisions:

- Cache sizes
- Associativity
- Number of levels
- Inclusive vs exclusive
- Shared vs private

No single optimal solution! It depends on the workload.

Datacenter CPU

- Large LLC
- Many cores
- High bandwidth

Mobile CPU

- Small caches
- Energy optimized

Closing

Key concepts:

- Multilevel AMAT
- Role of each cache level
- Inclusive vs exclusive
- Private vs shared caches
- System-level trade-offs

Next episode: cache coherence (why it is needed, MESI protocol and multicore behavior)